

Robotique Terrestre



COMPTE-RENDU DE TRAVAIL SUR LE TURTLEBOT

Tom HURARD, Antoine HUREAU et Amélie DUCHEMIN
Parcours SYSMER
SeaTech 2A

Professeur encadrant : M. Vincent HUGEL
École d'ingénieurs SeaTech Toulon

Table des matières

1	Descriptif de l'automate à états finis (FSM)	4
1.1	Structure générale	4
1.2	Liste des états et comportements associés	4
1.3	Logique des transitions	5
1.4	Résumé du flux d'exécution	5
2	Description des algorithmes utilisés	5
2.1	Gestion des mouvements : Lissage de vitesse	5
3	Perception : Analyse des données LIDAR	5
3.1	Filtrage et détection par seuillage	6
3.2	Stratégies de déplacement et décision	6
3.2.1	Navigation autonome	6
3.2.2	Manœuvre d'évitement séquentielle	6
4	Extensions et améliorations possibles	7
4.1	Navigation basée sur l'odométrie	7
4.2	Cartographie et Localisation	7
5	Conclusion	7
6	Annexes	8
6.1	Vidéo	8
6.2	Programmation et algorithme codé sous Python	8

Introduction

Contexte du projet

La robotique mobile autonome repose sur la capacité d'un système à percevoir son environnement, à l'analyser et à prendre des décisions de navigation en temps réel. Nous nous sommes intéressés au pilotage du robot TurtleBot, une plateforme standard pour l'apprentissage et le prototypage en robotique.

L'objectif principal était de concevoir et d'implémenter une intelligence de navigation permettant au robot d'évoluer en autonomie dans un environnement inconnu tout en assurant sa protection face aux obstacles.

Problématique et solution technique

La complexité de la gestion des comportements robotiques se trouve souvent dans la transition entre différentes tâches (avancer, tourner, s'arrêter, analyser). Pour répondre à ce défi, nous avons développé notre application en langage Python.

L'automate permet de décomposer le comportement du robot en "états" distincts (par exemple : Exploration, Évitement, Arrêt) et de définir des conditions de transition précises basées sur les données sensorielles.

Nous commencerons par piloter le robot à l'aide d'une manette puis nous le passerons en mode automatique au cours duquel le robot sera autonome.

Objectifs de l'application

Les objectifs spécifiques de ce développement sont les suivants :

Perception : Traiter les données brutes issues du télémètre laser (LIDAR) pour identifier les obstacles et les zones libres.

Décision : Implémenter une machine à états capable de basculer de manière fluide entre un comportement de croisière et des manœuvres d'évitement d'urgence.

Action : Commander les actionneurs (vitesse linéaire et angulaire) du TurtleBot en fonction de l'état actif pour assurer une navigation sûre.

Plan du document

Ce compte rendu détaille la démarche adoptée pour mener à bien ce projet. Nous présenterons dans un premier temps la structure de l'automate à états finis tel qu'il a été conçu et modifié. Nous détaillerons ensuite les algorithmes clés, notamment l'analyse des données LIDAR et les stratégies de déplacement. Enfin, nous aborderons les tests effectués (simulation et réel) ainsi que les extensions possibles pour améliorer la robustesse du système.

1 Descriptif de l'automate à états finis (FSM)

1.1 Structure générale

L'intelligence du robot est structurée autour d'un automate à états finis (FSM) qui orchestre les transitions entre le pilotage manuel et le mode autonome, ainsi que la gestion des collisions. L'automate garantit qu'à tout instant, le robot se trouve dans un état unique qui définit une action précise (avancer, reculer, tourner, attendre).

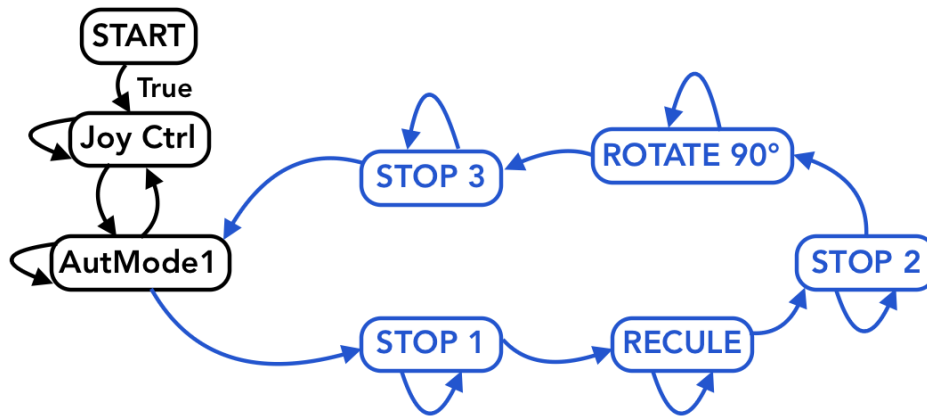


FIGURE 1 – Schéma graphique des transitions de l'automate.

1.2 Liste des états et comportements associés

L'automate est composé des états principaux suivants :

JoyControl (État Initial/Manuel) :

Le robot est piloté manuellement via le joystick. C'est l'état par défaut après l'initialisation (Start). Les vitesses linéaires et angulaires sont proportionnelles aux axes du joystick, avec un lissage (fonction `smooth_velocity`) pour éviter les à-coups mécaniques.

AutonomousMode1 (Croisière) :

Il s'agit du mode de déplacement autonome nominal. Le robot avance en ligne droite à une vitesse constante (fixée à $\frac{V_{max}}{2}$). Durant cet état, le système surveille en permanence les capteurs de contact (Bumpers).

Séquence d'évitement d'obstacle :

Lorsqu'une collision est détectée, l'automate enclenche une séquence stricte d'états pour se dégager :

- **Stop1 (Arrêt d'urgence)** : Arrêt immédiat des moteurs suite à l'impact. Une temporisation est lancée.
- **Recul (Dégagement)** : Le robot effectue une marche arrière (vitesse $-V_{max}$) pour s'éloigner physiquement de l'obstacle.
- **Stop2 (Stabilisation)** : Nouvel arrêt pour casser l'inertie avant d'entamer la rotation.
- **Rotation90 (Réorientation)** : Le robot tourne sur lui-même (vitesse angulaire positive) pour changer de cap.

- **Stop3 (Attente de reprise)** : Arrêt final de la manœuvre d'évitement. Le robot attend une confirmation ou la fin de la temporisation pour reprendre sa mission.

1.3 Logique des transitions

Les passages d'un état à un autre sont régis par trois types de conditions :

- **Commandes utilisateur (Joystick)** : Le passage de `JoyControl` à `AutonomousMode1` (et inversement) s'effectue par l'appui sur un bouton spécifique du joystick (bouton A). Le retour au mode autonome depuis la fin de la séquence d'évitement (**Stop3**) nécessite également une validation utilisateur.
- **Capteurs (Bumpers)** : La transition critique `AutonomousMode1` $\rightarrow$ `Stop1` est déclenchée par l'événement `BumperEvent.PRESSED`. Une variable d'état `obs_detect` passe à `True` dès qu'un choc est détecté.
- **Temporisations (Compteurs de cycles)** : Les états de la manœuvre d'évitement s'enchaînent automatiquement grâce à des compteurs incrémentés à chaque itération de la boucle de contrôle ('cpt1' à 'cpt5'). La boucle tournant à une fréquence de 10 Hz (soit une période $T = 0.1$ s), les seuils sont définis comme suit :
 - `Stop1` $\rightarrow$ `Recul` : après 10 cycles (soit 1 seconde).
 - `Recul` $\rightarrow$ `Stop2` : après 10 cycles.
 - `Stop2` $\rightarrow$ `Rotation90` : après 20 cycles (soit 2 secondes).
 - `Rotation90` $\rightarrow$ `Stop3` : après 10 cycles.

1.4 Résumé du flux d'exécution

Cette architecture permet de sécuriser le comportement du robot : en cas de choc, l'automate force l'interruption du mouvement en cours. Il impose une séquence de dégagement (arrêt, recul, rotation) avant de permettre, sous condition, la reprise du mode autonome.

2 Description des algorithmes utilisés

Cette section détaille la logique algorithmique implémentée pour transformer les données brutes des capteurs en commandes motrices cohérentes. Nous distinguons la gestion bas niveau (lissage) de l'intelligence de navigation (traitement LIDAR).

2.1 Gestion des mouvements : Lissage de vitesse

Afin de préserver la mécanique du robot et d'éviter les glissements de roues qui fausseraient l'odométrie, nous n'appliquons pas directement les consignes de vitesse brutes. Un algorithme de lissage (*Ramping*) est implémenté dans la méthode `smooth_velocity`.

À chaque itération de la boucle de contrôle, la vitesse réelle est mise à jour.

3 Perception : Analyse des données LIDAR

Le robot est équipé d'un télémètre laser retournant un message de type `sensor_msgs/LaserScan`. Le traitement est effectué dans le callback `processScan` afin de détecter la présence immédiate d'obstacles.

3.1 Filtrage et détection par seuillage

Contrairement à une analyse géométrique complexe, nous avons opté pour une approche robuste basée sur un seuillage de proximité. L'algorithme parcourt l'ensemble des mesures brutes (`data.ranges`) retournées par le LIDAR.

Une distance de sécurité est définie :

$$D_{\text{detect}} = 0.5 \text{ m} \quad (1)$$

Pour chaque point mesuré, l'algorithme vérifie deux conditions :

- La validité de la mesure.
- Si la distance mesurée est inférieure au seuil D_{detect} .

Dès qu'un point répond à ces critères, un drapeau global `self.obsdetect` est passé à `True`. Pour optimiser les ressources de calcul, la boucle de détection s'interrompt immédiatement après la première détection positive. Ce drapeau servira de déclencheur pour les transitions de la machine à états.

3.2 Stratégies de déplacement et décision

L'architecture décisionnelle repose sur une machine à états finis (FSM) qui gère les transitions entre le pilotage manuel, la navigation autonome et les manœuvres d'évitement.

3.2.1 Navigation autonome

Dans l'état `AutonomousMode1`, le comportement par défaut du robot est d'avancer en ligne droite à une vitesse constante ($V_{\text{max}}/3$). À chaque itération, la FSM vérifie l'état du capteur LIDAR (via le flag `self.obsdetect`) ou des bumpers.

3.2.2 Manœuvre d'évitement séquentielle

Si un obstacle est détecté (`obsdetect == True`), la machine à états quitte le mode autonome pour engager une séquence d'évitement réflexe stricte. Cette manœuvre se décompose en quatre états successifs, dont la durée est gérée par des compteurs d'itérations (`cpt`) :

1. **Arrêt d'urgence (Stop1)** : Le robot s'immobilise immédiatement pour éviter la collision.
2. **Recul (Recule)** : Le robot effectue une marche arrière à vitesse linéaire négative ($-V_{\text{max}}/3$) pendant une durée fixe (définie par `seuil = 10` itérations) pour s'éloigner de l'obstacle.
3. **Stabilisation (Stop2)** : Une courte pause est marquée pour dissiper l'inertie avant la rotation.
4. **Réorientation (Rotate)** : Le robot effectue une rotation sur place avec une vitesse angulaire $\omega_z = V_{\text{max}}/2$ pour changer de cap.
5. **Stabilisation finale (Stop3)** : Une dernière pause avant de redonner la main à l'état `AutonomousMode1`.

Cette stratégie garantit que le robot ne tente pas de contourner un obstacle de manière fluide, mais privilégie la sécurité en s'arrêtant et en changeant totalement de direction avant de reprendre sa route.

4 Extensions et améliorations possibles

Le système actuel, bien que fonctionnel pour des environnements simples, repose sur une approche réactive et des temporisations en boucle ouverte. Plusieurs pistes d'amélioration pourraient être envisagées pour accroître l'autonomie et la robustesse du robot.

4.1 Navigation basée sur l'odométrie

Actuellement, les rotations (ex : 90° après un obstacle) sont gérées par des compteurs de temps (boucles). Cette méthode est imprécise car elle dépend de la charge de la batterie et des frottements au sol.

- **Amélioration** : Utiliser les données du topic `/odom` pour asservir les rotations. L'automate passerait à l'état suivant non pas après un temps t , mais lorsque la différence angulaire mesurée atteint un seuil cible ($\Delta\theta \geq \frac{\pi}{2}$).
- **Intérêt** : Garantir des manœuvres géométriques précises indépendamment des perturbations extérieures.

4.2 Cartographie et Localisation

Le robot navigue actuellement "à l'aveugle" sans mémoriser son environnement.

- **Objectif** : Construire une carte 2D de la pièce en temps réel et permettre au robot de planifier une trajectoire globale d'un point A à un point B, plutôt que de naviguer au hasard.

5 Conclusion

Ce projet nous a permis de concevoir et d'implémenter une architecture de contrôle complète pour un robot mobile autonome, passant de la théorie des automates à la réalité matérielle du TurtleBot.

L'utilisation d'une Machine à États Finis (FSM) s'est révélée être une méthode structurante et efficace. Elle a permis de décomposer un problème complexe de navigation en comportements unitaires simples et testables (navigation de croisière, réflexe de recul, évitement actif). L'intégration des données du bumper et du LIDAR a conféré au robot une double couche de sécurité : une capacité d'anticipation grâce au laser et une sécurité ultime grâce aux capteurs de contact.

Les phases de validation, effectuées d'abord en simulation puis sur le robot réel, ont mis en lumière les défis inhérents à la robotique : gestion du bruit des capteurs, importance du calibrage des temporisations et ajustement des gains de vitesse pour garantir la stabilité du système.

En définitive, l'application développée constitue une base logicielle modulaire et robuste. Bien que limitée par une navigation "à l'aveugle" (sans cartographie), elle est parfaitement prédisposée à intégrer des briques technologiques plus avancées telles que la localisation SLAM ou la planification de trajectoire globale, ouvrant la voie à des missions de robotique de service plus complexes.

6 Annexes

6.1 Vidéo

Voici une compilation des différents comportements du robot : simulation Gazebo, à l'aide du Bumper et enfin à l'aide du LIDAR.

<https://youtu.be/6W-9P3nt3xU>

6.2 Programmation et algorithme codé sous Python

seatech-sm.py

```
#####
# imports
#####
import rospy
import math
from sensor_msgs.msg import Joy, LaserScan
from geometry_msgs.msg import Twist
from fsm import fsm
from kobuki_msgs.msg import BumperEvent

#####
# class RobotBehavior
#####
class RobotBehavior(object):
    #####
    # constructor, called at creation of instance
    #####
    def __init__(self, handle_pub, T):
        self.twist = Twist()
        self.twist_real = Twist()
        self.vreal = 0.0 # longitudinal velocity
        self.wreal = 0.0 # angular velocity
        self.vmax = 1.5
        self.wmax = 4.0
        self.previous_signal = 0
        self.button_pressed = False
        self.joy_activated = False
        self.pub = handle_pub
        self.T = T
        self.cpt = 0
        self.cpt1 = 0
        self.cpt2 = 0
        self.cpt3 = 0
        self.cpt4 = 0
        self.obsdetect = False

        self.fs = fsm([
```

```

("Start", "JoyControl", True),
("JoyControl", "AutonomousMode1",
 self.check_JoyControl_To_AutonomousMode1,
 self.DoAutonomousMode1),
("JoyControl", "JoyControl", self.KeepJoyControl,
 self.DoJoyControl),

("AutonomousMode1", "JoyControl",
 self.check_AutonomousMode1_To_JoyControl,
 self.DoJoyControl),
("AutonomousMode1", "Stop1",
 self.check_AutonomousMode1_To_Stop1,
 self.DoStartStop1),
("AutonomousMode1", "AutonomousMode1",
 self.KeepAutonomousMode1, self.DoAutonomousMode1),

("Stop1", "Recule", self.check_Stop1_To_Recule,
 self.DoStartRecule),
("Stop1", "Stop1", self.KeepStop1, self.DoKeepStop1),

("Recule", "Stop2", self.check_Recule_To_Stop2,
 self.DoStartStop2),
("Recule", "Recule", self.KeepRecule, self.DoKeepRecule),

("Stop2", "Rotate", self.check_Stop2_To_Rotate,
 self.DoStartRotate),
("Stop2", "Stop2", self.KeepStop2, self.DoKeepStop2),

("Rotate", "Stop3", self.check_Rotate_To_Stop3,
 self.DoStartStop3),
("Rotate", "Rotate", self.KeepRotate, self.DoKeepRotate),

("Stop3", "AutonomousMode1",
 self.check_Stop3_To_AutonomousMode1,
 self.DoAutonomousMode1),
("Stop3", "Stop3", self.KeepStop3, self.DoKeepStop3)
])

```

```

#####
# callback for joystick feedback
#####
def callback(self, data):
    self.twist.linear.x = self.vmax * data.axes[1]
    self.twist.linear.y = 0
    self.twist.linear.z = 0
    self.twist.angular.x = 0
    self.twist.angular.y = 0
    self.twist.angular.z = self.wmax * data.axes[4]

```

```
# for transition conditions of fsm
if not self.button_pressed:
    self.button_pressed = (self.previous_signal == 0 and
                           data.buttons[0] == 1)
self.previous_signal = data.buttons[0]

self.joy_activated = (abs(data.axes[1]) > 0.05 or
                     abs(data.axes[2]) > 0.05)

def processScan(self, data):
    if self.obsdetect:
        return

    dist_detect = 0.5 # Distance de securit (0.5 metres)

    for value in data.ranges:

        if not math.isnan(value) and value < dist_detect:
            rospy.loginfo("Obstacle Lidar detecte a : " +
                          str(value))
            self.obsdetect = True
            break

#####
# smoothing velocity function
#####
def smooth_velocity(self):
    accmax = 0.01
    accwmax = 0.05
    vjoy = 0.0
    wjoy = 0.0
    vold = 0.0
    wold = 0.0

    # filter twist
    vjoy = self.twist.linear.x
    vold = self.vreal
    deltav_max = accmax / self.T

    # vreal
    if abs(vjoy - self.vreal) < deltav_max:
        self.vreal = vjoy
    else:
        sign_ = 1.0
        if vjoy < self.vreal:
            sign_ = -1.0
        else:
            sign_ = 1.0
        self.vreal = vold + sign_ * deltav_max
```

```
# saturation
if self.vreal > self.vmax:
    self.vreal = self.vmax
elif self.vreal < -self.vmax:
    self.vreal = -self.vmax

# filter twist
wjoy = self.twist.angular.z
wold = self.wreal
deltaw_max = accwmax / self.T

# wreal
if abs(wjoy - self.wreal) < deltaw_max:
    self.wreal = wjoy
else:
    sign_ = 1.0
    if wjoy < self.wreal:
        sign_ = -1.0
    else:
        sign_ = 1.0
    self.wreal = wold + sign_ * deltaw_max

# saturation
if self.wreal > self.wmax:
    self.wreal = self.wmax
elif self.wreal < -self.wmax:
    self.wreal = -self.wmax

self.twist_real.linear.x = self.vreal
self.twist_real.angular.z = self.wreal

#####
# functions for fsm transitions
#####

def check_JoyControl_To_AutonomousMode1(self, fss):
    return self.button_pressed

def check_AutonomousMode1_To_JoyControl(self, fss):
    return self.joy_activated

def check_AutonomousMode1_To_Stop1(self, fss):
    if self.obsdetect == True:
        return True
    else:
        return False

def check_Stop1_To_Recule(self, fss):
    seuil = 10
```

```
        if self.cpt > seuil:
            return True
        else:
            return False

def KeepStop1(self, fss):
    return not self.check_Stop1_To_Recule(fss)

def DoKeepStop1(self, fss, value):
    self.cpt = self.cpt + 1
    pass

def check_Recule_To_Stop2(self, fss):
    seuil = 10
    if self.cpt1 > seuil:
        return True
    else:
        return False

def KeepRecule(self, fss):
    return not self.check_Recule_To_Stop2(fss)

def DoKeepRecule(self, fss, value):
    self.cpt1 = self.cpt1 + 1

def check_Stop2_To_Rotate(self, fss):
    seuil = 10
    if self.cpt2 > seuil:
        return True
    else:
        return False

def KeepStop2(self, fss):
    return not self.check_Stop2_To_Rotate(fss)

def DoKeepStop2(self, fss, value):
    self.cpt2 = self.cpt2 + 1

def check_Rotate_To_Stop3(self, fss):
    seuil = 10
    if self.cpt3 > seuil:
        return True
    else:
        return False

def KeepRotate(self, fss):
    return not self.check_Rotate_To_Stop3(fss)

def DoKeepRotate(self, fss, value):
```

```
        self.cpt3 = self.cpt3 + 1

def DoKeepStop3(self, fss, value):
    self.cpt4 = self.cpt4 + 1

def check_Stop3_To_AutonomousModel(self, fss):
    seuil = 10
    if self.cpt4 > seuil:
        return True
    else:
        return False

def KeepStop3(self, fss):
    return not self.check_Stop3_To_AutonomousModel(fss)

def KeepJoyControl(self, fss):
    return not self.check_JoyControl_To_AutonomousModel(fss)

def KeepAutonomousModel(self, fss):
    return not (self.check_AutonomousModel_To_JoyControl(fss) or
                self.check_AutonomousModel_To_Stop1(fss))

#####
# functions for instructions inside states of fsm
#####
def DoJoyControl(self, fss, value):
    self.button_pressed = False
    self.smooth_velocity()
    self.pub.publish(self.twist_real)
    pass

def DoAutonomousModel(self, fss, value):
    self.obsdetect = False
    self.cpt4 = 0
    self.button_pressed = False
    # go forward
    go_fwd = Twist()
    go_fwd.linear.x = self.vmax / 3.0
    self.pub.publish(go_fwd)
    pass

def DoStartStop1(self, fss, value):
    self.cpt = 0
    self.obsdetect = False
    go_fwd = Twist()
    go_fwd.linear.x = 0
    self.pub.publish(go_fwd)
    pass
```

```
def DoStartRecul(self, fss, value):
    self.cpt = 0
    self.cpt1 = 0
    go_fwd = Twist()
    go_fwd.linear.x = -self.vmax / 3.0
    self.pub.publish(go_fwd)
    pass

def DoStartStop2(self, fss, value):
    self.cpt1 = 0
    self.cpt2 = 0
    go_fwd = Twist()
    go_fwd.linear.x = 0
    self.pub.publish(go_fwd)
    pass

def DoStartRotate(self, fss, value):
    self.cpt2 = 0
    self.cpt3 = 0
    go_fwd = Twist()
    go_fwd.angular.z = self.wmax / 2.0
    self.pub.publish(go_fwd)
    pass

def DoStartStop3(self, fss, value):
    self.cpt3 = 0
    self.cpt4 = 0
    go_fwd = Twist()
    go_fwd.angular.z = 0
    self.pub.publish(go_fwd)
    pass

def processBump(self, data):
    rospy.loginfo("Collision %d", data.bumper)
    if self.obsdetect == False:
        if data.state == BumperEvent.PRESSED:
            self.obsdetect = True

#####
# main function
#####
if __name__ == '__main__':
    try:
        rospy.init_node('joy4ctrl')
        # real turtlebot2
        pub = rospy.Publisher('mobile_base/commands/velocity',
                               Twist, queue_size=10)
```

```
Hz = 10
rate = rospy.Rate(Hz)
T = 1.0 / Hz

MyRobot = RobotBehavior(pub, T)
rospy.Subscriber("joy", Joy, MyRobot.callback)
MyRobot.fs.start("Start")

rospy.Subscriber("/mobile_base/events/bumper", BumperEvent,
    MyRobot.processBump)
#Lidar
rospy.Subscriber("/scan", LaserScan, MyRobot.processScan)

# loop at rate Hz
while not rospy.is_shutdown():
    ret = MyRobot.fs.event("")
    rate.sleep()

except rospy.ROSInterruptException:
    pass
```